A Metacognition-Based Empathetic Smart Home

Jeong-O Heo
Department of Information Systems
Hanyang University
Seoul, South Korea

Jihun Shin
Department of Information Systems
Hanyang University
Seoul, South Korea

Seokjun Park
Department of Information Systems
Hanyang University
Seoul, South Korea

Junhyeong Kim
Department of Information Systems
Hanyang University
Seoul, South Korea

***Abstract*—This project designs a CLOiD decision prototype that adapts assistance to the resident's activity and intentions. A separate vision-language model (VLM) proposes assistance from recent observations and procedural assistance memory (PAM) structured as a finite-state machine (FSM). Local twin-home checks missing information and execution outcomes and carries out authorized tasks at times that limit disruption. In the demonstration, a person performs refrigerator door closure on behalf of CLOiD, ThinQ controls the air conditioner, and a virtual dishwasher washes the cup.**

ROLE ASSIGNMENTS (TILL THE MIDDLE OF THE TERM)

Roles	Name	Task description and etc.
User	Jihun Shin	Provide household scenarios, intentions, exceptions, and usability feedback.
Customer	Junhyeong Kim	Review the problem, scope, priorities, and acceptance criteria.
Software developer	Seokjun Park	Specify inputs, outputs, requirements, constraints, and verification cases; subsequently design, implement, and test.
Development manager	Jeong-O Heo	Coordinate schedules, progress, changes, integration, and deliverables.

I. INTRODUCTION

A refrigerator door needs to stay open while the resident retrieves ingredients. Once the resident moves to the counter, closing a door left open can help. PAM stores useful assistance and signs of transitions for each activity phase.

Metacognition checks which information is missing and whether outcomes differ from expectations. The agent can request a view from above when it cannot see inside a cup. Empathetic intelligence guides how and when to assist. The agent defers nonurgent questions while the resident is drinking.

The demonstration follows one resident through meal preparation, eating, and rest. The resident can check the outcome of each task and see why a task is waiting.

II. REQUIREMENTS

A. Common Behavior

The system shall compare execution candidates with waiting by considering expected outcomes, disruption, and delay. It shall record the basis for each decision and its outcome by execution route.

MEM-01. The system shall store activity phases, assistance conditions and actions, and transitions in FSM-structured PAM with versioned provenance.

MEM-02. The system shall use the day's fixed memory, recent observations, and task list for inference and distinguish action proposals, waiting, and response errors.

MEM-03. The system shall generate memory revisions from daily records, actual action outcomes, and resident feedback, validate their format, references, and allowed actions, and store them as the next day's version.

MEM-04. The system shall manage remembered routines separately from authorization and prioritize current intentions, refusals, and authorization changes in execution decisions.

B. Refrigerator Manipulation and Appliance Requirements

FRG-01. The system shall use timestamped appliance state and images to determine separately whether the refrigerator door is open and whether the resident is using it.

FRG-02. The system shall wait during active use, renewed approach, or uncertainty. When the door stays open and work continues at the counter, the system shall recheck the door area and the resident's authorization.

FRG-03. If the appliance provides no command to close the door, the system shall select direct CLOiD manipulation. During the demonstration, it shall identify the physical door in a phone instruction and receive the demonstrator's execution report.

FRG-04. The system shall link the instruction, executor, route, and outcome by task ID and record completion after confirming that the same physical door closed.

THQ-01. The system shall query registered physical appliances for supported functions and state, and record the time, source, and target ID.

THQ-02. The system shall check activity, current intentions, authorization, and current appliance state before applying an authorized air-conditioner setting.

THQ-03. The system shall distinguish ThinQ command acceptance from physical application and determine completion by reading back appliance state.

THQ-04. The system shall track goals, executions, and outcomes separately for ThinQ control, demonstrator manipulation, and virtual washing.

C. UC-01: Cup Cleaning

When the resident sets down a cup and leaves, the agent assesses whether to clean it. If the dishwasher is running, the agent retains the original goal until the next cycle.

D. Cup-Cleaning Functional Requirements

CUP-01. The system shall interpret the target cup and its usage context from input observations and distinguish information that cannot be determined.

CUP-02. The system shall generate an observation request identifying the information needed and the purpose of the request when cup-cleaning information is insufficient.

CUP-03. The system shall ask the resident when intention or authorization is required and incorporate the answer into its decision.

CUP-04. The system shall compare alternatives, including cleaning and waiting, using the information obtained and applicable authorization.

CUP-05. The system shall not virtually clear a cup that is in use, intended for reuse, or lacks the required cleaning authorization.

CUP-06. The system shall reconsider an unexecuted plan when relevant new information arrives before execution.

CUP-07. The system shall execute the selected virtual plan, including emptying remaining drink when needed, loading the dishwasher, starting a cycle, and washing, while distinguishing progress, success, and failure.

CUP-08. The system shall record observation requests, questions, answers, decisions, reasons, and results, and explain a decision from that history when asked.

CUP-09. The system shall report completion only after confirming the completion of the virtual wash cycle that contains the target cup.

CUP-10. The system shall confirm that use has ended and that required disposal is authorized before emptying remaining drink and loading the cup.

CUP-11. The system shall check the necessary availability conditions before loading or starting the dishwasher.

CUP-12. The system shall retain the original cup's unfinished goal, confirmed information, authorization, and waiting reason while the dishwasher is busy.

CUP-13. The system shall be able to search for other dishes while waiting and establish subsequent-cycle candidates through necessary observations, usage-intention checks, and authorization.

CUP-14. The system shall recheck availability after the current cycle finishes and resume washing prepared items in a subsequent cycle.

CUP-15. The system shall recheck each item's current conditions before the subsequent cycle and shall not repeatedly defer a prepared original goal solely because additional search or candidate answers remain unfinished.

III. DEVELOPMENT ENVIRONMENT

A. Platform and Tools

The demonstration connects one Android phone to twin-home on one Windows 11 laptop over a private Tailscale network. To protect images that may contain residents or their living space, access to the hub over this network is restricted to the registered demonstration phone. The phone camera provides CLOiD's view, and twin-home serves as the home hub. The vision-language model processes images and memory in a separate environment while the local hub manages appliance states and tasks.

- twin-home runs on Node.js 22 or later to handle events from the phone and appliances. TypeScript defines the shapes of state and commands.
- The phone's Chrome interface obtains its viewpoint through WebXR and sends images and state over WebSocket. three.js displays the virtual home and device state.
- Matter controls virtual devices. ThinQ provides physical appliance state and supported commands.
- twin-studio uses Blender 5.2 LTS and Python 3.11 or later to produce the home's GLB files and scene.json. twin-home loads the virtual home from these files.

GitHub tracks the manuscript, code, and home assets.

B. Development Cost

Project expenses are limited to GPU usage for running the model and model serving.

IV. SPECIFICATIONS

A. Assistance Proposals and Responses

twin-home sends recent observations, the day's PAM, and a task list to the model. It interprets the action index against the list sent with that request. An action index of -1 indicates waiting and creates no new goal. It records timeouts, unmatched responses, and out-of-range indices as errors. Existing goals remain active during waiting or response errors. [1]

B. Twin State and Agent Execution

twin-home tracks the locations of CLOiD and its targets in the virtual home. It synchronizes physical appliance states with ThinQ readbacks and updates virtual device states from execution results.

The agent expands the model's proposed action into steps. It checks each step's prerequisites, appliance state and capabilities, robot and target locations, and the resident's intent and authorization. If direct manipulation requires moving into position, it adds that move as a prerequisite. When a step is blocked, it records why and when to resume. It revises unstarted steps and their order when a resident response or the state changes. [2]

The agent executes each ready step through a supported route and updates the twin with the verified outcome before assessing subsequent steps. It allows one control step at a time per device. If the virtual dishwasher is running, it holds cup loading while an authorized air-conditioner adjustment can proceed separately. Execution IDs prevent duplicate actions.

C. Observation, Unknowns, and Questions: CUP-01–03

If a cup's interior is obscured, the agent leaves its contents unknown. When the contents affect the cleaning decision, it requests an image with the target, purpose, and viewing angle. It asks the resident when the target is ambiguous or when reuse intent or authorization is needed. It reuses valid answers and leaves unanswered items unknown.

D. Comparison and Pre-execution Checks: CUP-04–06

twin-home compares action with waiting by applying each candidate's tool effects to a copy of the current state. It excludes cups in use, intended for reuse, or lacking confirmed authorization. It rechecks the target, state, and authorization before dispatch and updates the plan if any condition has changed.

```
on_event(event):
  state = update_twin_state(event)
  if new_observation_window(event):
    request_proposal(window, day_memory, tasks)
  if is_model_result_or_timeout(event):
    proposal = interpret_or_record_error(event)
    if proposal.has_action:
      merge_goal(proposal)
  request_needed_information(state)
  preview_and_revise_plans(state)
  for step in ready_steps_in_order():
    route = find_available_supported_route(step)
    if route and recheck_and_reserve(step, route):
      dispatch(step, route)
```

E. Washing and Completion: CUP-07, 09–11

If a cup authorized for cleaning contains a drink, the agent confirms disposal permission before emptying it. It loads the cup when the dishwasher is idle and has room for it. The virtual cycle starts once a cup is loaded and the door is closed. A failed step holds its successors. When the virtual dishwasher starts, twin-home creates a cycle ID and links the loaded cup IDs. The cycle-ending event carries the cycle ID and each cup's result. Loading, starting, and washing are progress states. It records completion when the linked cycle finishes and confirms that the target cup was washed successfully. It records an unrelated cycle ending, a wash failure, or an unknown result under its own status.

F. Waiting and the Next Cycle: CUP-12–15

When the dishwasher is running, the original cup retains its goal, information, authorization, waiting reason, and resume condition. Additional search begins from observed information; each new item requires its own end-of-use and authorization checks. Authorization for the original cup is not copied to new items. After the current cycle ends, space and availability are rechecked. A prepared original goal is not repeatedly delayed by further search or unanswered candidates.

```
on_dishwasher_event(event):
  if is_wash_result(event):
    confirm_matching_cup_and_cycle(event)
  if dishwasher_is_running():
    inspect_additional_items_if_useful()
    return retain_waiting_goals()
  if not loading_conditions_hold():
    return record_waiting_reason()
  ready = recheck_each_prepared_target()
  batch = fit_original_then_other_items(ready)
  if batch:
    queue_empty_load_and_start_steps(batch)
```

G. Records and Explanations: CUP-08

Target IDs link observations, requests, answers, choices, authorization, execution, and completion judgments. Packet IDs, task lists, and memory versions trace model decisions. The system records discrepancies between predicted and observed outcomes and uses that evidence to explain completion, deferral, or failure when asked.

H. Refrigerator Closure and Phone Work Instructions

The agent pairs refrigerator state with phone images using the door ID and observation time (FRG-01 and FRG-02). It waits while the resident takes out ingredients. It considers closing the door when the resident continues working at the counter and the area around the open door is clear. It rechecks state and authorization before dispatch. If the resident has already closed the door, it ends the task without issuing an instruction. If the resident returns, the door area is obscured, or observations stop, it waits and cancels any unexecuted instruction.

twin-home reads the appliance profile to check whether it can read door state and whether it can send a closure command (FRG-03, THQ-01). It considers appliance control when the command is available and selects direct CLOiD manipulation when the profile shows that it is unsupported. A failed query leaves the task waiting for a fresh profile. For direct manipulation, the phone shows the task ID, target door, action, and reason. The demonstrator checks the active instruction and door area, closes the physical door as CLOiD's hands, and reports what they did.

When the demonstrator reports execution, twin-home reads back the state of the same physical door through ThinQ. If the appliance does not expose door state, it checks post-execution phone images. Evidence of closure marks the task complete and updates the twin display. Insufficient evidence leaves the result unverified. Task and door IDs link the instruction, actual executor, route, and closure evidence (FRG-04).

I. Air-Conditioner Control

When the resident begins resting in the living room, the agent checks whether the preferred temperature is within both the current authorization and the range supported by the appliance. It sends a ThinQ command if both conditions hold and the physical setpoint differs from the target. It waits if the resident refuses or the device state is unknown (THQ-02).

twin-home records completion after reading the target setpoint back from the same air conditioner. Device rejection is a

failure. An accepted command without confirmed application remains unverified (THQ-03). Each task has a separate record of its route, executor, and outcome (THQ-04).

J. Experience and Memory Updates

PAM stores the initial state and state list in JSON. Each state has an ID and description, rules that pair assistance conditions with actions, and next entries for destinations and transition conditions. twin-home links each version to its effective date, input records, and validation results. twin-home uses the same memory version throughout the day. At day's end, the previous memory, sampled observations, execution outcomes, and resident feedback inform revisions to states, assistance rules, and transition conditions. A version that passes format, state-reference, and action-list checks takes effect the next day. If validation fails, twin-home keeps the last valid version and logs the error. It records direct feedback and inferred reactions with separate sources. The model weights stay fixed as the JSON memory changes.

REFERENCES

[1] D. Lee et al., *An Agentic Framework for Proactive Robots with Procedural Assistance Memory*, arXiv:2610.21211, 2026. [Online]. Available: https://arxiv.org/abs/2610.21211

[2] Y. Jung, H. Kim, K.-D. Suh, and J.-M. Park, *Human-Centered Dynamic Service Scheduling Approach in Multi-Agent Environments*, Applied Sciences, vol. 12, no. 21, Art. no. 10850, 2022, doi: 10.3390/app122110850.